```
!pip install requests langgraph boto3 awscli

!aws configure

from langgraph.graph import StateGraph
from typing import TypedDict, Dict, Any, List
import requests, json, random, time, boto3

# AWS CloudWatch setup for logs
log_group = "/LangGraph/TaskRouting"
log_stream = f"run_{int(time.time())}"
logs_client = boto3.client("logs", region_name="us-east-1")

try:
    logs_client.create_log_group(logGroupName=log_group)
except logs_client.exceptions.ResourceAlreadyExistsException:
    pass
logs_client.create_log_stream(logGroupName=log_group, logStreamName=log_stream)

sequence_token = None
def log(msg: str):
    global sequence_token
    kwargs = {
        "logGroupName": log_group,
        "logStreamName": log_stream,
        "logEvents": [{"timestamp": int(time.time() * 1000), "message": msg}]
    }
    if sequence_token:
        kwargs["sequenceToken"] = sequence_token
    response = logs_client.put_log_events(**kwargs)
    sequence_token = response.get("nextSequenceToken")

# -------------------------
# 1️⃣ Define State Schema
# -------------------------
class GraphState(TypedDict):
    lambda_data: Dict[str, Any]
    task_type: str
    result: str

LAMBDA_API_URL = "https://2ooay23h0c.execute-api.us-east-1.amazonaws.com/default/LangGraphStaticData"

# -------------------------
# 2️⃣ Fetch Data from Lambda
# -------------------------
def fetch_lambda_data(state: GraphState):
    response = requests.get(LAMBDA_API_URL)
    data = response.json()
    if "body" in data:
        data = json.loads(data["body"])
    state["lambda_data"] = data
    # Randomly assign task type to simulate dynamic routing
    state["task_type"] = random.choice(["admin", "user", "moderator"])
    log(f"Fetched data for task type: {state['task_type']}")
    return state

# -------------------------
# 3️⃣ Task-specific Nodes
# -------------------------
def handle_admin_task(state: GraphState):
    users = [u for u in state["lambda_data"]["users"] if u["role"] == "admin"]
    state["result"] = f"Admin tasks completed for {len(users)} users."
    log(state["result"])
    return state

def handle_user_task(state: GraphState):
    users = [u for u in state["lambda_data"]["users"] if u["role"] == "user"]
    state["result"] = f"User tasks completed for {len(users)} users."
    log(state["result"])
    return state

def handle_moderator_task(state: GraphState):
    users = [u for u in state["lambda_data"]["users"] if u["role"] == "moderator"]
    state["result"] = f"Moderator tasks completed for {len(users)} users."
    log(state["result"])
    return state

# -------------------------
# 4️⃣ Routing Function
# -------------------------
def route_tasks(state: GraphState):
```

```
    task = state["task_type"]
    if task == "admin":
        return "handle_admin_task"
    elif task == "user":
        return "handle_user_task"
    else:
        return "handle_moderator_task"

# -------------------------
# 5 Build the Graph
# -------------------------
graph = StateGraph(GraphState)
graph.add_node("fetch_lambda_data", fetch_lambda_data)
graph.add_node("handle_admin_task", handle_admin_task)
graph.add_node("handle_user_task", handle_user_task)
graph.add_node("handle_moderator_task", handle_moderator_task)

graph.add_conditional_edges(
    "fetch_lambda_data",
    route_tasks,
    {
        "handle_admin_task": "handle_admin_task",
        "handle_user_task": "handle_user_task",
        "handle_moderator_task": "handle_moderator_task",
    },
)

graph.set_entry_point("fetch_lambda_data")
compiled_graph = graph.compile()

from concurrent.futures import ThreadPoolExecutor

def run_simulation(i):
    result = compiled_graph.invoke({"lambda_data": {}, "task_type": "", "result": ""})
    print(f"Run {i}: {result['result']}")
    return result["result"]

# Simulate 5 concurrent agent executions
with ThreadPoolExecutor(max_workers=5) as executor:
    results = list(executor.map(run_simulation, range(1, 6)))

print("\nAll simulation results:")
print(results)
```

```
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (2.32.4)
Requirement already satisfied: langgraph in /usr/local/lib/python3.12/dist-packages (0.6.9)
Requirement already satisfied: boto3 in /usr/local/lib/python3.12/dist-packages (1.40.47)
Requirement already satisfied: awscli in /usr/local/lib/python3.12/dist-packages (1.42.47)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests) (3.4.3)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests) (2025.8.3)
Requirement already satisfied: langchain-core>=0.1 in /usr/local/lib/python3.12/dist-packages (from langgraph) (0.3.77)
Requirement already satisfied: langgraph-checkpoint<3.0.0,>=2.1.0 in /usr/local/lib/python3.12/dist-packages (from langgraph) (
Requirement already satisfied: langgraph-prebuilt<0.7.0,>=0.6.0 in /usr/local/lib/python3.12/dist-packages (from langgraph) (0
Requirement already satisfied: langgraph-sdk<0.3.0,>=0.2.2 in /usr/local/lib/python3.12/dist-packages (from langgraph) (0.2.9)
Requirement already satisfied: pydantic>=2.7.4 in /usr/local/lib/python3.12/dist-packages (from langgraph) (2.11.9)
Requirement already satisfied: xxhash>=3.5.0 in /usr/local/lib/python3.12/dist-packages (from langgraph) (3.5.0)
Requirement already satisfied: botocore<1.41.0,>=1.40.47 in /usr/local/lib/python3.12/dist-packages (from boto3) (1.40.47)
Requirement already satisfied: jmespath<2.0.0,>=0.7.1 in /usr/local/lib/python3.12/dist-packages (from boto3) (1.0.1)
Requirement already satisfied: s3transfer<0.15.0,>=0.14.0 in /usr/local/lib/python3.12/dist-packages (from boto3) (0.14.0)
Requirement already satisfied: docutils<=0.19,>=0.18.1 in /usr/local/lib/python3.12/dist-packages (from awscli) (0.19)
Requirement already satisfied: PyYAML<6.1,>=3.10 in /usr/local/lib/python3.12/dist-packages (from awscli) (6.0.3)
Requirement already satisfied: colorama<0.4.7,>=0.2.5 in /usr/local/lib/python3.12/dist-packages (from awscli) (0.4.6)
Requirement already satisfied: rsa<4.8,>=3.1.2 in /usr/local/lib/python3.12/dist-packages (from awscli) (4.7.2)
Requirement already satisfied: python-dateutil<3.0.0,>=2.1 in /usr/local/lib/python3.12/dist-packages (from botocore<1.41.0,>=1
Requirement already satisfied: langsmith<1.0.0,>=0.3.45 in /usr/local/lib/python3.12/dist-packages (from langchain-core>=0.1->l
Requirement already satisfied: tenacity!=8.4.0,<10.0.0,>=8.1.0 in /usr/local/lib/python3.12/dist-packages (from langchain-core>
Requirement already satisfied: jsonpatch<2.0.0,>=1.33.0 in /usr/local/lib/python3.12/dist-packages (from langchain-core>=0.1->l
Requirement already satisfied: typing-extensions<5.0.0,>=4.7.0 in /usr/local/lib/python3.12/dist-packages (from langchain-core>
Requirement already satisfied: packaging<26.0.0,>=23.2.0 in /usr/local/lib/python3.12/dist-packages (from langchain-core>=0.1->
Requirement already satisfied: ormsgpack>=1.10.0 in /usr/local/lib/python3.12/dist-packages (from langgraph-checkpoint<3.0.0,>=
Requirement already satisfied: httpx>=0.25.2 in /usr/local/lib/python3.12/dist-packages (from langgraph-sdk<0.3.0,>=0.2.2->lang
Requirement already satisfied: orjson>=3.10.1 in /usr/local/lib/python3.12/dist-packages (from langgraph-sdk<0.3.0,>=0.2.2->lan
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.12/dist-packages (from pydantic>=2.7.4->langgra
Requirement already satisfied: pydantic-core==2.33.2 in /usr/local/lib/python3.12/dist-packages (from pydantic>=2.7.4->langgrap
Requirement already satisfied: typing-inspection>=0.4.0 in /usr/local/lib/python3.12/dist-packages (from pydantic>=2.7.4->langg
Requirement already satisfied: pyasn1>=0.1.3 in /usr/local/lib/python3.12/dist-packages (from rsa<4.8,>=3.1.2->awscli) (0.6.1)
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx>=0.25.2->langgraph-sdk<0.3.0,>=0.2
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx>=0.25.2->langgraph-sdk<0.3
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx>=0.25.2->langgra
Requirement already satisfied: jsonpointer>=1.9 in /usr/local/lib/python3.12/dist-packages (from jsonpatch<2.0.0,>=1.33.0->lang
Requirement already satisfied: requests-toolbelt>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from langsmith<1.0.0,>=0.3
Requirement already satisfied: zstandard>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from langsmith<1.0.0,>=0.3.45->lar
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil<3.0.0,>=2.1->botocore<
Requirement already satisfied: sniffio>=1.1 in /usr/local/lib/python3.12/dist-packages (from anyio->httpx>=0.25.2->langgraph-sc
```

```
AWS Access Key ID [****************SBWJ]: AKIAVSX4Z23PGE3ZSBWJ
AWS Secret Access Key [****************QiEG]: 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
Default region name [us-east-1]: us-east-1
Default output format [None]:
Run 1: Admin tasks completed for 1 users.
Run 2: Moderator tasks completed for 1 users.
Run 4: User tasks completed for 1 users.
Run 5: Moderator tasks completed for 1 users.
Run 3: Admin tasks completed for 1 users.

All simulation results:
['Admin tasks completed for 1 users.', 'Moderator tasks completed for 1 users.', 'Admin tasks completed for 1 users.', 'User ta
```

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.

Start coding or generate with AI.